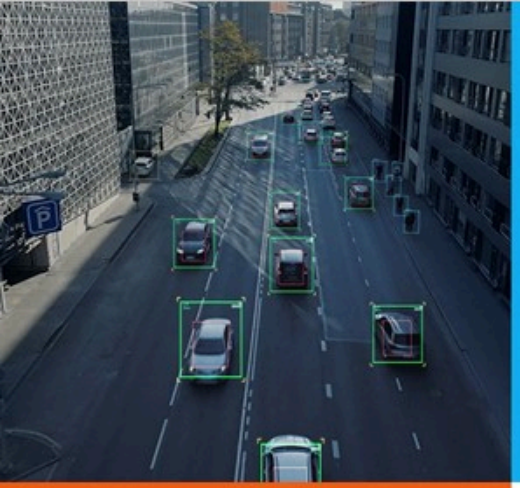


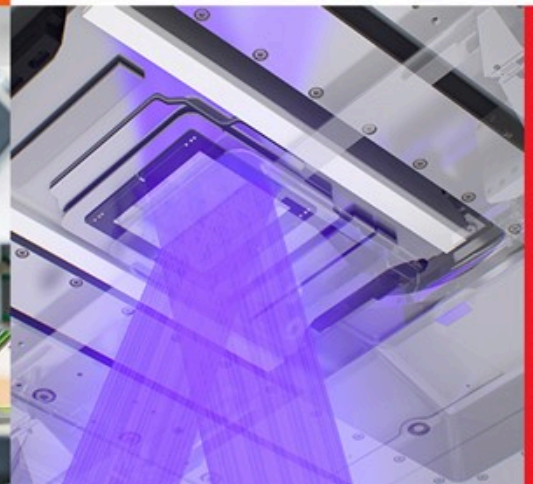
MECHATRONICS

MATHWARE

ASSEMBLY

SIOUX TECHNOLOGIES

ELECTRONICS



SOFTWARE

C++ Training

Modules

Why Modules

```
int work(int x) {
    return x *2
}
int main() {
    return work(10);
}
```

Converting cpp file with compiler & linker to executable.

```
g++ -c main.cpp
g++ main.o -o main
```

© Sioux Technologies | Confidential

Linker does the heavy lifting.

```
//lib.cpp
int work(int x) {
    return x *2
}
```

```
//main.cpp
int work(int x); //JUST TRUST ME
int main() {
    return work(10);
}
```

© Sioux Technologies | Confidential

Multiple C++ source files

- Source files compiled into Translation Units (TU)
- Name (symbol) is declared in every TU it is used in
- Linker finds 1 definition accros TUs. Pastes the symbol location into other TUs.

© Sioux Technologies | Confidential

```cpp
//lib.cpp
int work(int x, int y) {
    return x * y * 2;
}
```

```cpp
//main.cpp
int work(int x); //might have lied actually
int main() {
    return work(10);
}
```

```
<source>:3:(.text+0xa): undefined reference to `work(int)'
```

© Sioux Technologies | Confidential

Header files try to fix having to avoid declaring everything

```
//lib.h
int work(int x)
```

```
//lib.cpp
int work(int x) {
    return x * 2;
}
```

```
//main.cpp
#include "lib.h"
int main() {
    return work(10);
}
```

© Sioux Technologies | Confidential

Linker resolves symbols in step 3.

If .h/.cpp mismatch -> undefined symbol

The linker must be able to find a link for each global symbol.

Linker goes looking for a symbol that does not exist.

```
g++ -c lib.cpp
g++ -c main.cpp
g++ main.o lib.o -o main
```

© Sioux Technologies | Confidential

One definition Rule (ODR)

- Each name must be declared in every TU it is used
- Only one defintion is allowed in any one TU
- Cannot identify the one true definition, wont guess.
  Even if identical, linker script might make them end up in other memory.

© Sioux Technologies | Confidential

```
struct Point
{
    int x, y;
}
```

```
#include "point.h"
struct Rectangle
{
    Point tl, br;
}
```

```
#include "point.h"
#include "rectangle.h"
int main(){
    Point p{0, 0};
    Rectangle r{p,{1,1}}
}
```

© Sioux Technologies | Confidential

After preprocessor

```
struct Point
{
    int x, y;
}
struct Point
{
    int x, y;
}
struct Rectangle
{
    Point tl, br;
}
int main(){
    Rectangle r{{0,0},{1,1}}
}
```

ODR violation, redefinition of 'struct Point'

© Sioux Technologies | Confidential

if not defined preprocessor directive

```
//point.h
#ifndef POINT_H
#define POINT_H
struct Point
{
    int x, y;
}
#endif
```

© Sioux Technologies | Confidential

- #pragma once is a bit of a mess and sometimes cannot tell files apart.
- Say the same file is twice in your project (thirdparty dependency) #pragma once could implode (compiler specific)
  - File size
  - Modification time
  - File content

© Sioux Technologies | Confidential

```
struct Rectangle
{
    Point tl, br;
}
```

```
#include "point.h"
#include "rectangle.h"
int main(){
    Point p{0, 0};
    Rectangle r{p,{1,1}}
}
```

Does this compile?

© Sioux Technologies | Confidential

- You cannot tell if a header is 'complete' before including it in some TU and even then its difficult to tell.
- Headers are only compiled if included into some cpp file

© Sioux Technologies | Confidential

What should not be in a header(msft guidelines):

- built-in type definitions at namespace or global scope
- Non-inline function definitions
- Non-const variable definitions
- Aggregate definitions
- Unnamed namespaces
- Using directives

© Sioux Technologies | Confidential

built-in type definitions at namespace or global scope

define directives are bad practices in headers

NOT scoped to header

Polutes the whole TU.

```
using int = MyInt;
```

© Sioux Technologies | Confidential

Non-inline function definitions

```
//lib.h
struct Point{
};
void printPoint(Point const&){}
```

```
//lib.cpp
#include "lib.h"
```

```
//main.cpp
#include "lib.h"
```

Multiple defintions, obvious ODR violation

© Sioux Technologies | Confidential

Unnamed namespaces

```
//lib.h
namespace
{
    void printPoint(Point const&) {

    }
}
```

- Binary bloat, each TU file gets its own copy
  Extremely harmful for flash space.
- No ODR violation

© Sioux Technologies | Confidential

Using directives

```
//lib.h
using namespace std;
```

- Pollutes the global namespace
  - No scope limitation to 'this' header
- Increased chanse of name collision

© Sioux Technologies | Confidential

Modules

Improvement and eventual replacement of header files

Consume external code:

- Package components, encapsulate implementations
- Designed to co-exist but minimalize reliance on the preprocessor

© Sioux Technologies | Confidential

Goal

- Componentization
- Improve compilation speed
- Isolation from macros
- Reduce oppertunities for violating the ODR

© Sioux Technologies | Confidential

Conceptional idea

```
lib.ccpm ---> main.cpp ---> main.o----------> main
         |                            |
         |                            |
         ---> lib.o -------------------
```

Some entity that can be used to import from by main.cpp
but also be turned into a lib.o file.

© Sioux Technologies | Confidential

```
//lib.h
struct Point
{
    int x, y;
}
```

```
//lib.ccp
void doWork(Point const& p){}
```

```
//lib.ccpm
export module lib;
export struct Point
{
    int x, y;
}
export Point doWork(Point const& p){}
```

© Sioux Technologies | Confidential

Every module consists of one or more Module Units (MU)

- Special kind of TU
- Not exposed outside of the module
- Either interface of implementation unit
- Can be broken down into partitions.
  MUs are compiled, generating a Built Module Interface (BMI)

© Sioux Technologies | Confidential

Interface unit

```
export module myModule
export int foo();
```

Implementation unit

No export keyword.

```
module myModule
int foo() { return 69;}
```

© Sioux Technologies | Confidential

A module unit be broken down into partitions

```
//geometry.ccpm
export module geometry;
export import :circle;
```

```
//geometry-circle.ccpm
export module geometry:cricle;
import :helpers;
```

```
//geometry-helpers.ccpm
module geometry:helpers;
```

© Sioux Technologies | Confidential

What it really looks like

Externally visible as a 'module'

```
lib.ccpm ----------> main.cpp ---> main.o----------> main
| lib_part.ccpm  |                          |
|  |             |                          |
----> lib.pcm-------> lib.o -------------------
```

© Sioux Technologies | Confidential

Types of module units

- Primary module interface unit
- Module implementation unit
- Module partition interface unit
- Internal module partition unit

© Sioux Technologies | Confidential

Primary module interface unit

Interface module units without a partition

Need one of those. Mandatory.

```
export module module_name;
```

© Sioux Technologies | Confidential

Module implementation unit

Implementation module units without a partition:

```
module module_name
```

Useful for splitting interface/implementation

Implicitly performs

```
import module_name
```

defines the declarations, .cpp file if you will

© Sioux Technologies | Confidential

Module partition interface unit.

Interface module units with a partition:

```
export module module_name:partition_name;
```

Tool for separating large interfaces into multiple files

MUST be exported by the primary module interface unit

```
export module module_name;
export import:partition_name;
```

© Sioux Technologies | Confidential

Internal module partition unit

Implementation module units with a partition

```
module module_name:partition_name;
```

Useful for utility functions

Not part of the external interface of its module.

© Sioux Technologies | Confidential

Fragments

private vs public fragment.

```
export module myModule
export void foo(){}
void helper(){}
export void bar(){helper();}
```

```
export module myModule
export void foo();
export void bar();
module : private;
void foo(){}
void helper(){}
void bar(){helper();}
```

Primary module interface unit only

© Sioux Technologies | Confidential

Global module fragment

```
export module myModule;
#include <vector>
export void printSize(std::vector<int> const& vec){}
```

vector exported?

```
module;
#include <vector>
export module myModule;
export void printSize(std::vector<int> const& vec){}
```

© Sioux Technologies | Confidential

Header units

```
import 'header.h'
```

- TU formed by synthesizing an importable header
- All declaration are implicitly exported
- Legacy, porting

© Sioux Technologies | Confidential

```
//foo_legacy.h
#pragma once
#define FOO_VERSION 42
inline void foo(){}
```

```
//foo.cppm
export module foo;
export import "foo_legacy.h"; // header unit
export bool check_version(int version){
    return version == FOO_VERSION;
}
```

macro is firewalled (not exported) outside of foo.cppm

© Sioux Technologies | Confidential

STL header units

C++20 defines importable C++ library headers

```
#include <vector>
#include <map>
```

```
import <vector>;
import <map>;
```

© Sioux Technologies | Confidential

C++23 exposes the full STL as a module:

```
import std;
```

- std exposes all symbols in the std namespace
- std.compat for C global namespace decls. e.g. printf
- macros are not exposed (assert from cassert)

© Sioux Technologies | Confidential

Traditional Headers vs modules

Recap this example

```
//rectangle.h
struct Rectangle
{
    Point tl, br;
}
```

© Sioux Technologies | Confidential

```
//point.h
struct Point
{
    int x, y;
}
```

```
#include "point.h"
#include "rectangle.h"
int main(){
    Point p{0, 0};
    Rectangle r{p,{1,1}}
}
```

© Sioux Technologies | Confidential

```cpp
//point.cppm
export module point;
export struct Point
{
    int x, y;
}
```

```cpp
//rectangle.cppm
export module rectangle;
export import point;
export struct Rectangle
{
    Point tl, br;
}
```

© Sioux Technologies | Confidential

- It is safe to switch the order of point and rectangle
- rectangle is a 'complete' TU, cant accidently compile

```
import point;
import rectangle;
int main(){
    Point p{0, 0};
    Rectangle r{p,{1,1}}
}
```

© Sioux Technologies | Confidential

built-in type definitions at namespace or global scope

```
#pragma once
#include <cstdint>
#define int int64_t
```

```
module;
#define int int64_t
export module myModule;
```

Probably still a bad practise.

© Sioux Technologies | Confidential

Non-inline function definitions

Non-const variable definitions

```
void doSomething(){}
int variable;
```

Definitions can be part of a module and do not violate ODR.

```
export module myModule;
export int variable;
export void doSomething() {
}
```

© Sioux Technologies | Confidential

Unnamed namespace

```
export module myModule;
namespace
{
    export void doSomething(){}
}
```

Cannot export symbol due to internal linkage. compiler error.

© Sioux Technologies | Confidential

Using directives

```
export module myModule;
using namespace std;
```

Does not leak out. Only local to the module

No global namespace pollution.

© Sioux Technologies | Confidential

```
#define SECRET 42
namespace Private
{
    inline int secret() {return SECRET;}
}
```

```
//UserFacing.h
using Pimpl = shared_ptr<struct UserFacingImpl>;
class UserFacing
{
public:
    UserFacing();
    int getNumber() const;
private:
    Pimpl m_pimpl;
};
```

© Sioux Technologies | Confidential

```cpp
//UserFacing.cpp
#include "UserFacing.h"
#include "Private.h"

struct UserFacingImpl
{ int number = Private :secret(); };

UserFacing :UserFacing()
: m_pimpl(new UserFacingImpl()) {}

int UserFacing :getNumber() const
{ return m_pimpl->number; }
```

Pimpl, trying to hide private information from header file into cpp file.

© Sioux Technologies | Confidential

Same thing with modules

```
module;
#define SECRET 42 //dont leak define
export module Private;
namespace Private {
export inline int secret() { return SECRET; }
}
```

© Sioux Technologies | Confidential

```
export module UserFacing;
import Private; //dont re-export
struct UserFacingImpl
{ const int number = Private :secret(); };
export class UserFacing
{
public:
UserFacing() : m_pimpl(std :make_shared<UserFacingImpl>()) {}
int getNumber() const
{
    return m_pimpl->number;
}
private:
    std :shared_ptr<UserFacingImpl> m_pimpl;
};
```

© Sioux Technologies | Confidential

Lets write a module export a symbol and import

ex1.cpp

© Sioux Technologies | Confidential

Compile-time performance

One of the selling points

- hello_world.cpp: Just needs . not a lot of code
- mix.cpp: Requires including 9 standard headers. lots of code.,
- Hello world: 0.87s 3.43s (including all headers)
- mix: 2.2s 3.53s
- Notice convergence on 3.5s.

© Sioux Technologies | Confidential

- Hello world: 0.32s 0.62s (import all) 0.08s (import std)
- mix: 0.77s 0.99s (import all) 0.44s (import std)
  guideline: use import std.

© Sioux Technologies | Confidential

"But I could achieve this before with XYZ"

- Modules are standardized, accessible by every c+ developper
- Processed only once, into an efficient binary representation (BMI)
- Modules provide strong isolation, no polution

© Sioux Technologies | Confidential

Build systems are lagging. biggest challange is dependency scanning

```
export module square; //exporting some module
export import rectangle; // re-exporting rectangle
```

```
{
"primary-output": "square.o",
"provides": [{
"compiled-module-path": "square.pcm",
"logical-name": "square"
}],
"requires": [{
"logical-name": "rectangle"
}]
}
```

© Sioux Technologies | Confidential

CMake 3.28: Native support for named modules

```
cmake_minimum_required(VERSION 3.28 FATAL_ERROR)

set(CMAKE_CXX_STANDARD 20)

project(ModulesExample CXX)
add_library(Lib)
target_sources(Lib
PUBLIC FILE_SET modules TYPE CXX_MODULES FILES lib.cppm
PRIVATE lib.impl.cpp
)
add_executable(Main main.cpp)
target_link_libraries(Main PRIVATE Lib)
```

Works with Ninja and Visual Studio generators

RIP makefiles

CMake 3.30 for header module files.

© Sioux Technologies | Confidential

```
[1/10] Scanning lib.cppm for CXX dependencies
[2/10] Scanning main.cpp for CXX dependencies
[3/10] Scanning lib.impl.cpp for CXX dependencies
[4/10] Generating CXX dyndep file CMakeFiles/Lib.dir/CXX.dd
[5/10] Generating CXX dyndep file CMakeFiles/Main.dir/CXX.dd
[6/10] Building CXX object CMakeFiles/Lib.dir/lib.cppm.o
[7/10] Building CXX object CMakeFiles/Lib.dir/lib.impl.cpp.o
[8/10] Building CXX object CMakeFiles/Main.dir/main.cpp.o
[9/10] Linking CXX static library libLib.a
[10/10] Linking CXX executable Main
```

© Sioux Technologies | Confidential

Templates

```
//primary module math.cppm
export module Math;
export import :Add;
export import :Sub;
```

```
//add.cppm/sub.cppm
export module Math:Add;

export template <typename T>
T add(T a, T b) {
    return a + b + 101; // Change this later to (a + b + 0) to test
}
```

© Sioux Technologies | Confidential

```cpp
//usesadd.cpp
import Math;
#include <iostream>

void run_add_test() {
    std::cout << "Addition Result: " << add(10, 5) << std::endl;
}
```

```cpp
//usessub.cpp
import Math;
#include <iostream>

void run_sub_test() {
    std::cout << "Subtraction Result: " << subtract(10, 5) << std::endl;
}
```

© Sioux Technologies | Confidential

Some cpp file used to stich everything together

```
//main.cppp
void run_add_test();
void run_sub_test();

int main() {
    run_add_test();
    run_sub_test();
    return 0;
}
```

© Sioux Technologies | Confidential

```
cmake_minimum_required(VERSION 3.28)
project(ModuleTest LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 23)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_SCAN_FOR_MODULES ON)
add_library(MathModule)
target_sources(MathModule
  PUBLIC
    FILE_SET CXX_MODULES FILES
      math/math.cppm
      math/add.cppm
      math/sub.cppm
)
add_executable(app main.cpp usesadd.cpp usessub.cpp)
target_link_libraries(app MathModule)
```

© Sioux Technologies | Confidential

Maybe a change in sub.cppm would not have lead to a recompile of usesadd.cpp ?

```
[1/11] Scanning /source/math/add.cppm for CXX dependencies
[2/11] Generating CXX dyndep file CMakeFiles/MathModule.dir/CXX.dd
[3/10] Generating CXX dyndep file CMakeFiles/app.dir/CXX.dd
[4/9] Building CXX object CMakeFiles/MathModule.dir/math/add.cppm.o
[5/9] Building CXX object CMakeFiles/MathModule.dir/math/math.cppm.o
[6/9] Linking CXX static library libMathModule.a
[7/9] Building CXX object CMakeFiles/app.dir/usesadd.cpp.o
[8/9] Building CXX object CMakeFiles/app.dir/usessub.cpp.o
[9/9] Linking CXX executable app
```

- You can see expected nr of steps decreasing when CMake realises sub is not changed
- usessub.cpp is recompiled

© Sioux Technologies | Confidential

- This makes sense because Math module is imported, not the Sub part module.
- The speed up is in the binary representation of the template (part of the BMI)
  - Textual parsing avoided in each .cpp file
  - Textual parsing once when compiling BMI

© Sioux Technologies | Confidential

Adding Modules to existing projects:

- Not rewrite everything at once
- Build system bump is probably required.
- Look into PCH?
- Header units: bridge legacy and modern code
- Not everything works.
- Macro heavy headers will probably break.
- Self-contained components.
- Chicken and egg:
  - users complain tooling not there
  - tooling complains, users are not asking for it

© Sioux Technologies | Confidential

Built Module Interfaces

Current implementations are underwhelming:

- Difficult to utilize
- Not an information hiding mechanism
- Not portable

That doesn’t have to be forever:

- BMIs could be shipped independently
- Shared libraries could be type-safe and self-descriptive
- Cross-compiler compatibility could appear

© Sioux Technologies | Confidential

We bring **high-tech** to life